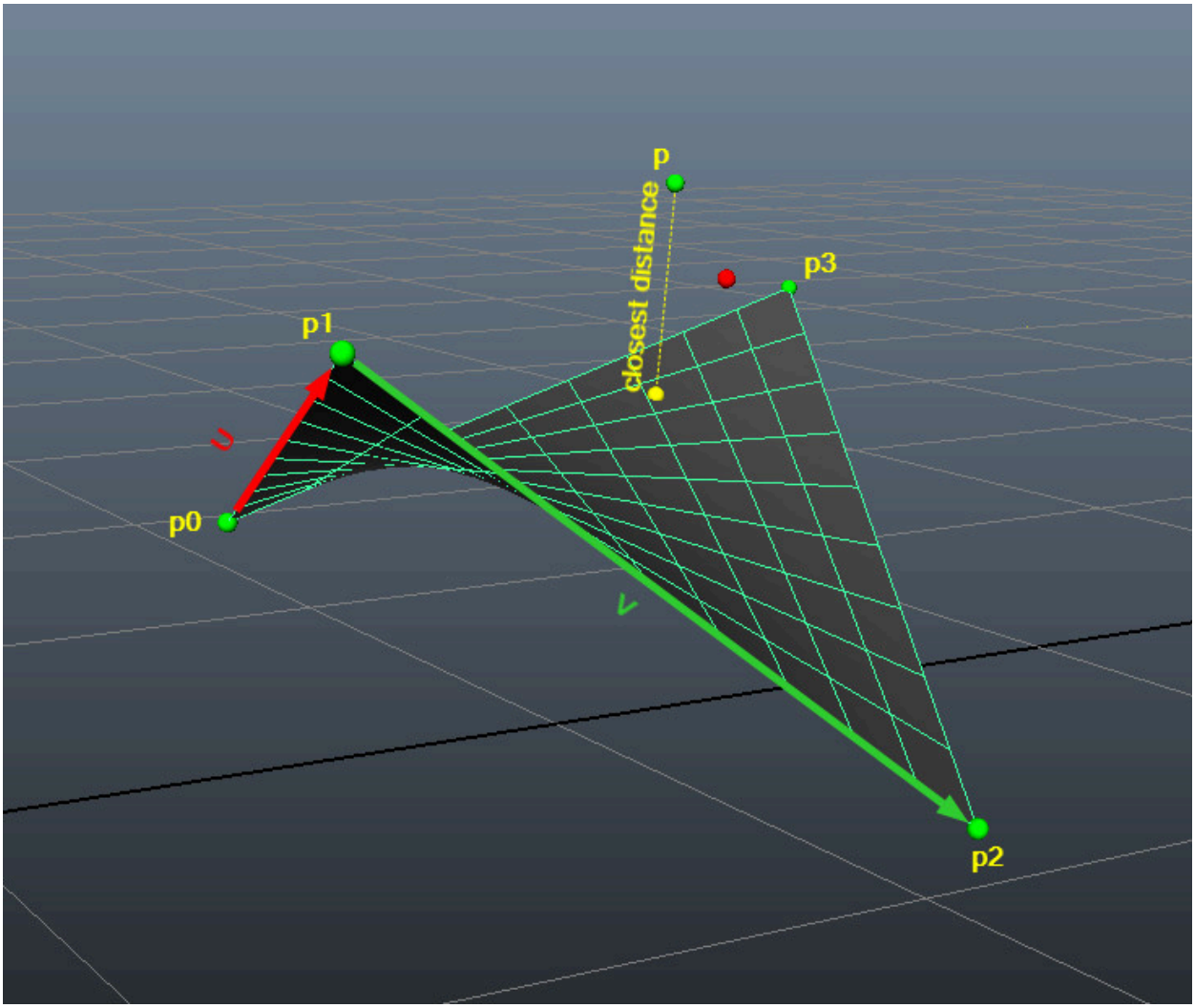
Get the closest point to a plane defined by four vertices in python

Asked 10 years, 7 months ago Modified 10 years, 7 months ago Viewed 3k times

3

I need to write a python script which will find the closest point to a plane defined by 4 vertices (given sequentially using the right hand rule). As a bonus it would be great to get the UV coordinates of the closest point.




```
Arrows along the edges show UV direction

p0 = [1.15, 0.62, -1.01]
p1 = [1.74, 0.86, -0.88]
p2 = [1.79, 0.40, -1.46]
p3 = [0.91, 0.79, -1.84]
p =  [1.17, 0.94, -1.52]
RESULT SHOULD BE APPROX:
Closest Point: 1.116 0.705 -1.527
Closest U Value: 0.164
Closest V Value: 0.682
```

Here's the code i wrote to solve my problem. Please let me know how i can improve it/make it faster.

```python
import numpy as np

# Define constants
LARGE_VALUE=99999999.0
SMALL_VALUE=0.00000001
SUBSAMPLES=10.0

def closestPointOnLineSegment(a,b,c):
    ''' Given two points (a,b) defining a line segment and a query point (c)
        return the closest point on that segment, the distance between
        query and closest points, and a u value derived from the results
    '''

    # Check if c is same as a or b
    ac=c-a
    AC=np.linalg.norm(ac)
    if AC==0.:
        return c,0.,0.

    bc=c-b
    BC=np.linalg.norm(bc)
    if BC==0.:
        return c,0.,1.


    # See if segment length is 0
    ab=b-a
    AB=np.linalg.norm(ab)
    if AB == 0.:
        return a,0.,0.

    # Normalize segment and do edge tests
    ab=ab/AB
    test=np.dot(ac,ab)
    if test < 0.:
        return a,AC,0.
    elif test > AB:
        return b,BC,1.

    # Return closest xyz on segment, distance, and u value
    p=(test*ab)+a
    return p,np.linalg.norm(c-p),(test/AB)




def surfaceWalk(e0,e1,p,v0=0.,v1=1.):
    ''' Walk on the surface along 2 edges, for each sample segment
        look for closest point, recurse until the both sampled edges
        are smaller than SMALL_VALUE
    '''

    edge0=(e1[0]-e0[0])
    edge1=(e1[1]-e0[1])
    len0=np.linalg.norm(edge0*(v1-v0))
    len1=np.linalg.norm(edge1*(v1-v0))

    vMin=v0
    vMax=v1
    closest_d=0.
    closest_u=0.
    closest_v=0.
    ii=0.
    dist=LARGE_VALUE

    for i in range(int(SUBSAMPLES)+1):
```

```
        v=v0+((v1-v0)*(i/SUBSAMPLES))
        a=(edge0*v)+e0[0]
        b=(edge1*v)+e0[1]

        closest_p,closest_d,closest_u=closestPointOnLineSegment(a,b,p)

        if closest_d < dist:
            dist=closest_d
            closest_v=v
            ii=i

    # If both edge lengths <= SMALL_VALUE, we're within our precision treshold
so return results
    if len0 <= SMALL_VALUE and len1 <= SMALL_VALUE:
        return closest_p,closest_d,closest_u,closest_v

    # Threshold hasn't been met, set v0 anf v1 limits to either side of
closest_v and keep recursing
    vMin=v0+((v1-v0)*(np.clip((ii-1),0.,SUBSAMPLES)/SUBSAMPLES))
    vMax=v0+((v1-v0)*(np.clip((ii+1),0.,SUBSAMPLES)/SUBSAMPLES))
    return surfaceWalk(e0,e1,p,vMin,vMax)




def closestPointToPlane(p0,p1,p2,p3,p,debug=True):
    ''' Given four points defining a quad surface (p0,p1,p2,3) and
        a query point p. Find the closest edge and begin walking
        across one end to the next until we find the closest point
    '''

    # Find the closest edge, we'll use that edge to start our walk
    c,d,u,v=surfaceWalk([p0,p1],[p3,p2],p)
    if debug:
        print 'Closest Point:     %s'%c
        print 'Distance to Point: %s'%d
        print 'U Coord:           %s'%u
        print 'V Coord:           %s'%v

    return c,d,u,v



p0 = np.array([1.15, 0.62, -1.01])
p1 = np.array([1.74, 0.86, -0.88])
p2 = np.array([1.79, 0.40, -1.46])
p3 = np.array([0.91, 0.79, -1.84])
p =  np.array([1.17, 0.94, -1.52])
closestPointToPlane(p0,p1,p2,p3,p)


Closest Point:     [ 1.11588876  0.70474519 -1.52660706]
Distance to Point: 0.241488104197
U Coord:           0.164463481066
V Coord:           0.681959858995
```

python numpy 3d scipy

Share Improve this question Follow

edited Sep 17, 2013 at 17:40

asked Sep 16, 2013 at 22:31

Fnord

5,585 ● 5 ● 33 ● 50

2 A plane defined by 4 points in 3D?? – gg349 Sep 16, 2013 at 22:45

1 Sounds like you're trying to do a change of basis. You're new basis will be U, V and the normal of the plane. You can do this with a little linear algebra. – Bi Rico Sep 16, 2013 at 22:51

1 Answer

Sorted by: Highest score (default)

9

A plane is not defined by 4 points, it is defined by only 3. In the example you have given, p2 is not contained in the plane defined by the other 3 points. So the question you are asking is ill defined. Ignoring the point p2, you would do something like:

```
p0 = np.array([1.15, 0.62, -1.01])
p1 = np.array([1.74, 0.86, -0.88])
p2 = np.array([1.79, 0.40, -1.46])
p3 = np.array([0.91, 0.79, -1.84])

p =  np.array([1.17, 0.94, -1.52])

u = p1 - p0
v = p3 - p0
# vector normal to plane
n = np.cross(u, v)
n /= np.linalg.norm(n)

p_ = p - p0
dist_to_plane = np.dot(p_, n)
p_normal = np.dot(p_, n) * n
p_tangent = p_ - p_normal

closest_point = p_tangent + p0
coords = np.linalg.lstsq(np.column_stack((u, v)), p_tangent)[0]

>>> dist_to_plane
0.11587377639539007
>>> closest_point
array([ 1.21810711,  0.84032937, -1.55432496])
>>> p0 + coords[0] * u + coords[1] * v
array([ 1.21810711,  0.84032937, -1.55432496])
>>> coords
array([ 0.40821569,  0.7197506 ])
```

`dist_to_plane` is the distance from `p` to the plane defined by `p0, p1, p3`, `closest_point` is the point in the plane closest to `p`, and coords holds the coordinates of that point relative to the vectors `u = p1 - p0` and `v = p3 - p0`.

Share Improve this answer Follow

edited Sep 16, 2013 at 23:46

answered Sep 16, 2013 at 23:03

Jaime
66.7k • 19 • 126 • 161

1 @BiRico I do indeed, I actually had it corrected in the code that I ran for the output, but didn't paste the correction here. Thanks! – Jaime Sep 16, 2013 at 23:47

@Jaime Thanks! See the image i uploaded above, the little red ball is your result, the one i'm looking for is the yellow ball on the surface. I might not have used the right terminology to describe my

problem, so hopefully the pic will help! :) – Fnord Sep 17, 2013 at 17:28

2 @Fnord, Jamie answered the question you asked. If the question you asked isn't the question you meant to ask, pick Jamie's answer for this question and ask a new question. – Bi Rico Sep 17, 2013 at 19:53

2 @Fnord Your "plane" seems to be a hyperbolic paraboloid, is that what you are after? – Jaime Sep 17, 2013 at 20:16

@Jaime yes. Thank you for your invaluable contribution by answering my question here: stackoverflow.com/questions/18858448/… – Fnord Sep 20, 2013 at 17:37